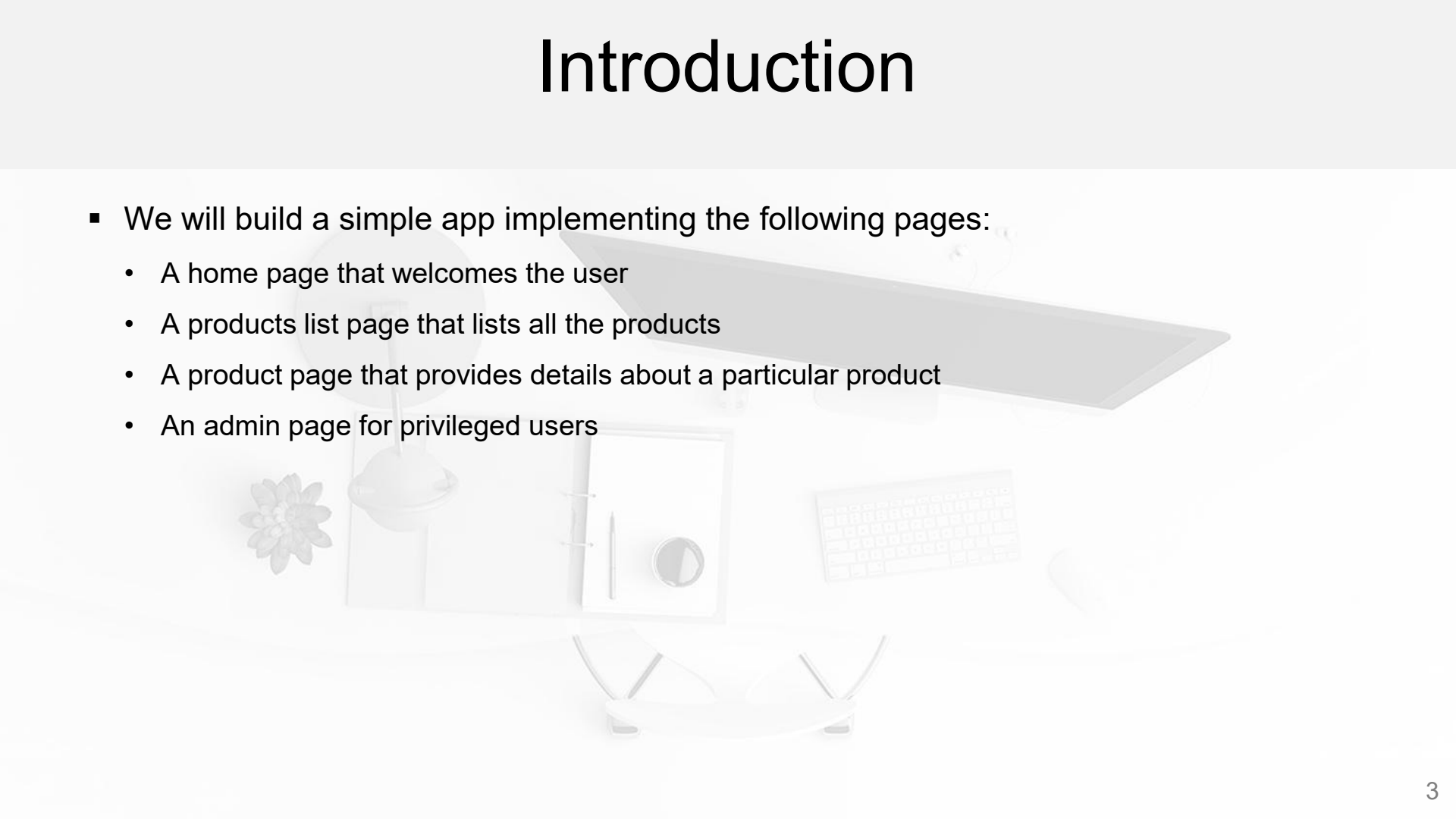
Advanced Web Programming

Ung Văn Giàu
Email: giau.ung@eiu.edu.vn

Routing with React Router

Introduction

- We will build a simple app implementing the following pages:
  - A home page that welcomes the user
  - A products list page that lists all the products
  - A product page that provides details about a particular product
  - An admin page for privileged users



Introduction

- We will cover the following topics:
  - Introducing React Router
  - Declaring routes
  - Creating navigation
  - Using nested routes
  - Using route parameters
  - Creating an error page
  - Using index routes
  - Using search parameters
  - Navigating programmatically
  - Using form navigation
  - Implementing lazy loading

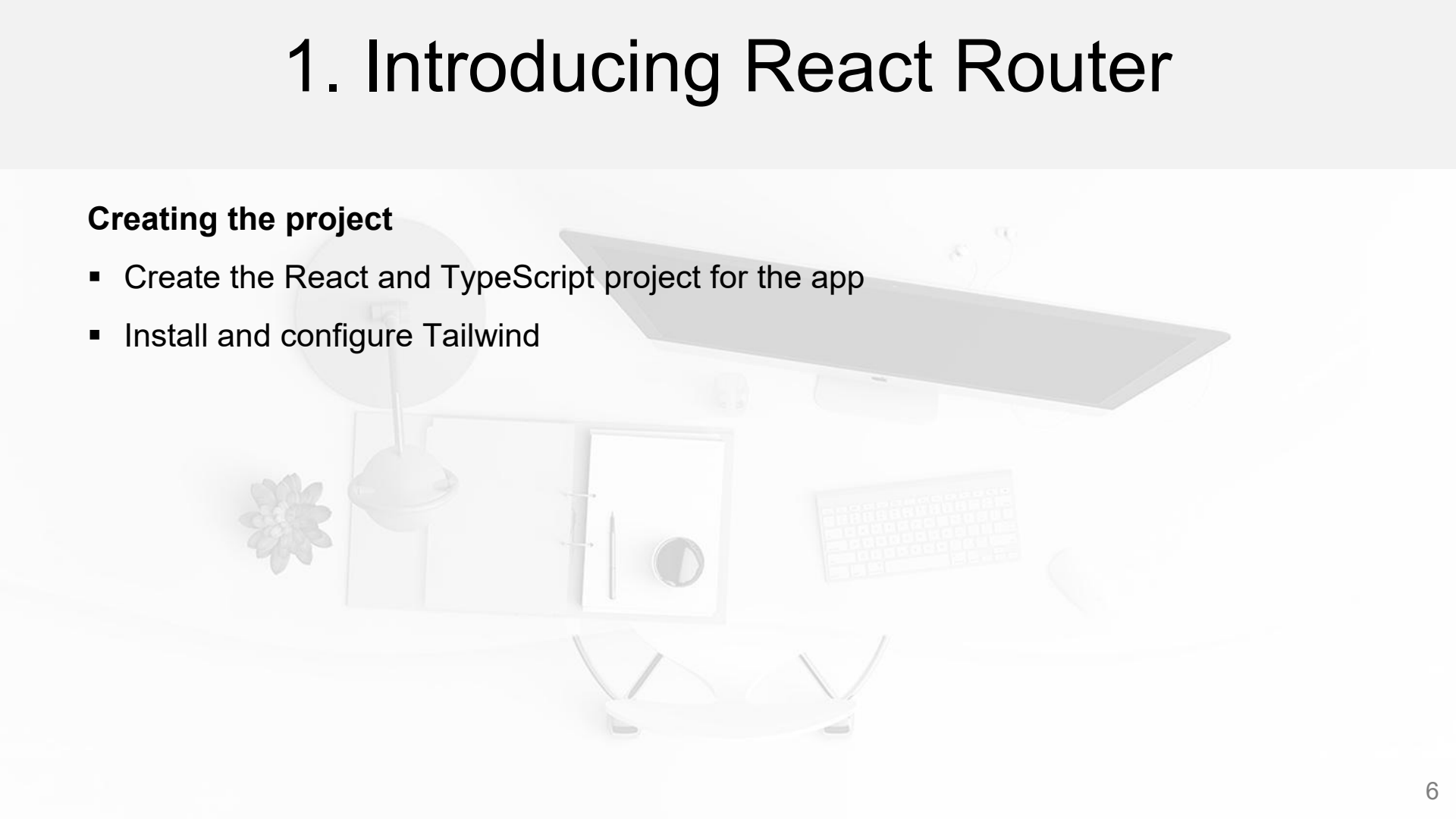
Technical requirements

- Browser
- Node.js and npm
- Visual Studio Code

1. Introducing React Router

Creating the project

- Create the React and TypeScript project for the app
- Install and configure Tailwind



1. Introducing React Router

Understanding React Router

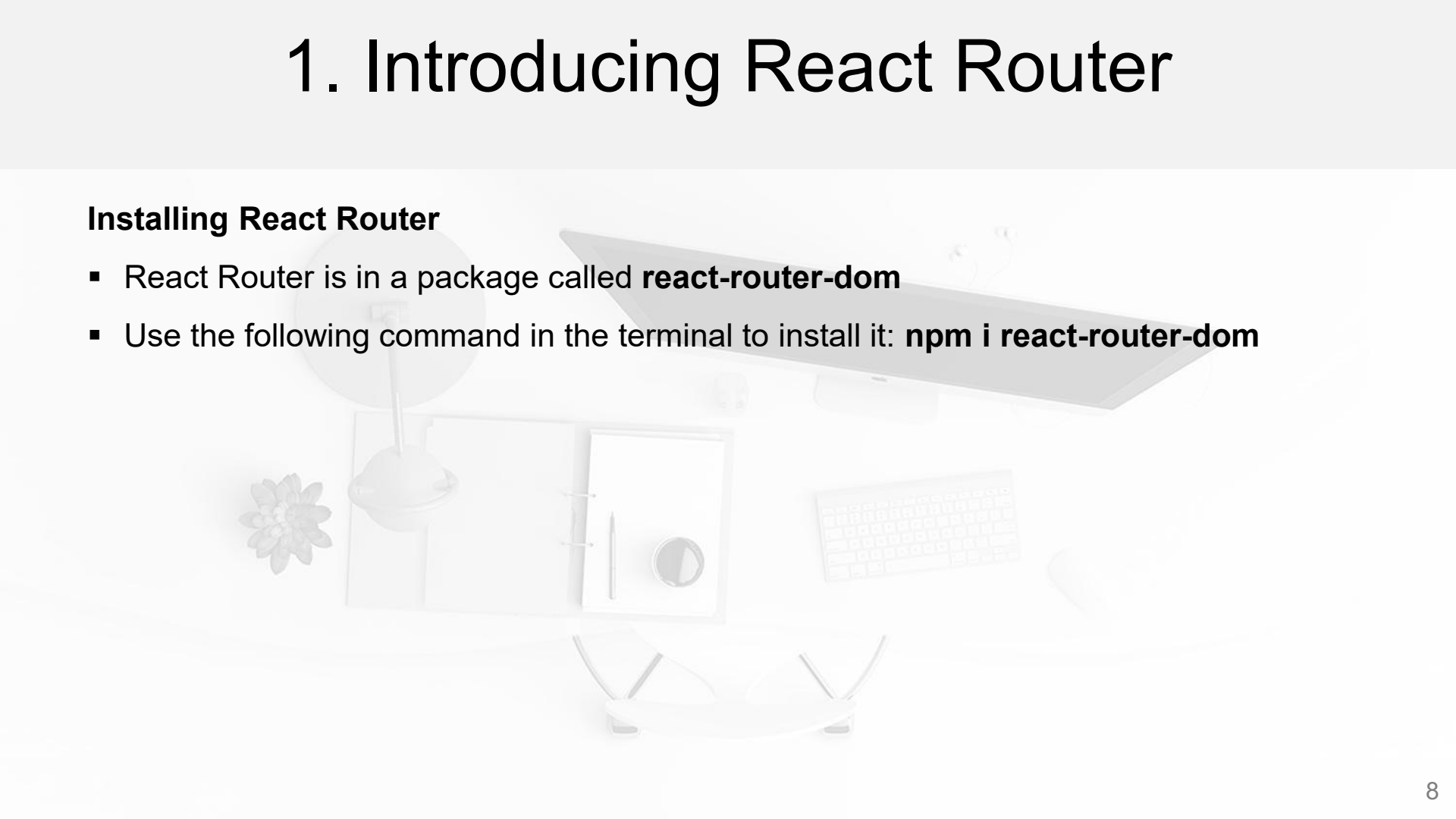
- React Router is **a routing library** for React apps.
- A **router** is responsible for **selecting what to show** in the app **for a requested path**.

  For example, it defines what components to render when a path of /products/6 is requested.

1. Introducing React Router

Installing React Router

- React Router is in a package called **react-router-dom**
- Use the following command in the terminal to install it: **npm i react-router-dom**



2. Declaring routes

Creating the products list page

- The products list page component will contain the list of the React tools in the app.
- Carry out the following steps:
  - Start by creating the data source for the page. Create a folder called **data** in the **src** folder and then a file called **products.ts** within **data**.
  - Copy the content from an attached file on EIU Moodle into **products.ts**
  - Create a folder for all the page components in the **src** folder called **pages**. Create a file called **ProductsPage.tsx** in the **pages** folder for the products list page component.

2. Declaring routes

Creating the products list page

- Using **Array.map** is common practice for JSX looping logic.
- Carry out the following steps:
  - Add the following **import** statement into **ProductsPage.tsx**

    import { products } from '../data/products';
  - Create the **ProductsPage** component

```
export function ProductsPage() {
  return (
    <div className="text-center p-5 text-xl">
      <h2 className="text-base text-slate-600">
        Here are some great tools for React
      </h2>
      <ul className="list-none m-0 p-0">
        {products.map((product) => (
          <li key={product.id} className="p-1 text-base textslate-800">
            {product.name}
          </li>
        ))}
      </ul>
    </div>
  );
}
```

2. Declaring routes

Recap

Creating the products list page

- **React requires the key prop on elements in a loop** to update the corresponding DOM elements efficiently.
- The value of the **key prop must be unique** and stable within the array, so we have used the product ID.

2. Declaring routes

Understanding React Router's router

- A **router** in React Router is a component that **tracks** the browser's **URL** and **performs navigation**. Several routers are available in React Router, and the one recommended for web applications is called a **browser router**.
- The **createBrowserRouter** function creates a browser router.
- **createBrowserRouter requires an argument containing all the routes** in the application.
- A **route contains** a **path** and **what component to render** when the app's browser address matches that path.

2. Declaring routes

Understanding React Router's router

- The following code snippet creates a router with two routes:

```
const router = createBrowserRouter([
  {
    path: 'some-page',
    element: <SomePage />,
  },
  {
    path: 'another-page',
    element: <AnotherPage />,
  }
]);
```

2. Declaring routes

Understanding React Router's router

- The router returned by **createBrowserRouter** is passed to a **RouterProvider** component and placed high up in the React component tree.

```
const root = ReactDOM.createRoot(
  document.getElementById('root') as HTMLElement
);
root.render(
  <React.StrictMode>
    <RouterProvider router={router} />
  </React.StrictMode>
)
```

2. Declaring routes

Declaring the products route

- Carry out the following steps:
  - Create a file called **Routes.tsx** in the **src** folder containing the following **import** statements:

```
import {
  createBrowserRouter,
  RouterProvider,
} from 'react-router-dom';
import { ProductsPage } from './pages/ProductsPage';
```

2. Declaring routes

Declaring the products route

- Carry out the following steps:
  - Add the following component underneath the **import** statements to define the router with a products route:

```
const router = createBrowserRouter([
  {
    path: 'products',
    element: <ProductsPage />,
  },
]);
```

2. Declaring routes

Declaring the products route

- Carry out the following steps:
  - Still in **Routes.tsx**, create a component called Routes under the router

```
export function Routes() {
  return <RouterProvider router={router} />;
}
```

  - Open the **index.tsx** file and add an **import** statement for the **Routes** component we just created beneath the other **import** statements:

```
import { Routes } from './Routes';
```

2. Declaring routes

Declaring the products route

- Carry out the following steps:
  - Render **Routes** instead of App as the top-level component

```
root.render(
  <React.StrictMode>
    <Routes />
  </React.StrictMode>
);
```

  - Run the app.
  - Change the browser URL to http://localhost:3000/products.

2. Declaring routes

Recap

Declaring the products route

- In a web app, **routes** in React Router **are defined using createBrowserRouter**
- **Each route has a path and a component** to render when the browser's URL matches that path
- The **router returned** from createBrowserRouter **is passed into a RouterProvider component**, which should **be placed high up** in the component tree

3. Creating navigation

React Router comes with components called **Link** and **NavLink**, which provide navigation.

Using the Link component

- Carry out the following steps:
  - Create a file for the app header called **Header.tsx** in the **src** folder containing the following **import** statements:

```
import { Link } from 'react-router-dom';
import logo from './logo.svg';
```

3. Creating navigation

Using the Link component

- Carry out the following steps:
  - Create the **Header** component

```
export function Header() {
  return (
    <header className="text-center text-slate-50
      bg-slate-900 h-40 p-5">
      <img
        src={logo}
        alt="Logo"
        className="inline-block h-20"
      />
      <h1 className="text-2xl">React Tools</h1>
      <nav></nav>
    </header>
  );
}
```

3. Creating navigation

Using the Link component

- Carry out the following steps:
  - Create a link inside the **nav** element:

```
<nav>
  <Link
    to="products"
    className="text-white no-underline p-1"
  >
      Products
  </Link>
</nav>
```

  - Open **Routes.tsx** and add an import statement for the **Header** component

```
import { Header } from './Header';
```

Creating navigation

Using the Link component

- Carry out the following steps:
  - In the **router** definition, add a root path that renders the **Header** component:

```
const router = createBrowserRouter([
  {
    path: '/',
    element: <Header />,
  },
  {
    path: 'products',
    element: <ProductsPage />,
  },
]);
```

  - In the running app, change the browser address to the root of the app.

Creating navigation

Using the NavLink component

- React Router's **NavLink** is like a Link element but allows it to **be styled differently when active**.
- Carry out the following steps to replace **Link** with **NavLink** in the app header:
  - Open **Header.tsx** and change the **Link** references to **NavLink**

```
import { NavLink } from 'react-router-dom';
...
export function Header() {
  return (
    <header ...>
      ...
      <nav>
        <NavLink
          to="products"
          className="..."
        >
          Products
        </NavLink>
      </nav>
    </header>
  );
}
```

Creating navigation

Using the NavLink component

- The **className** prop on the **NavLink** component **accepts a function** that can be used to **conditionally style** it, depending on whether its page is **active**.
- Carry out the following steps:
  - Update the **className** attribute to the following:

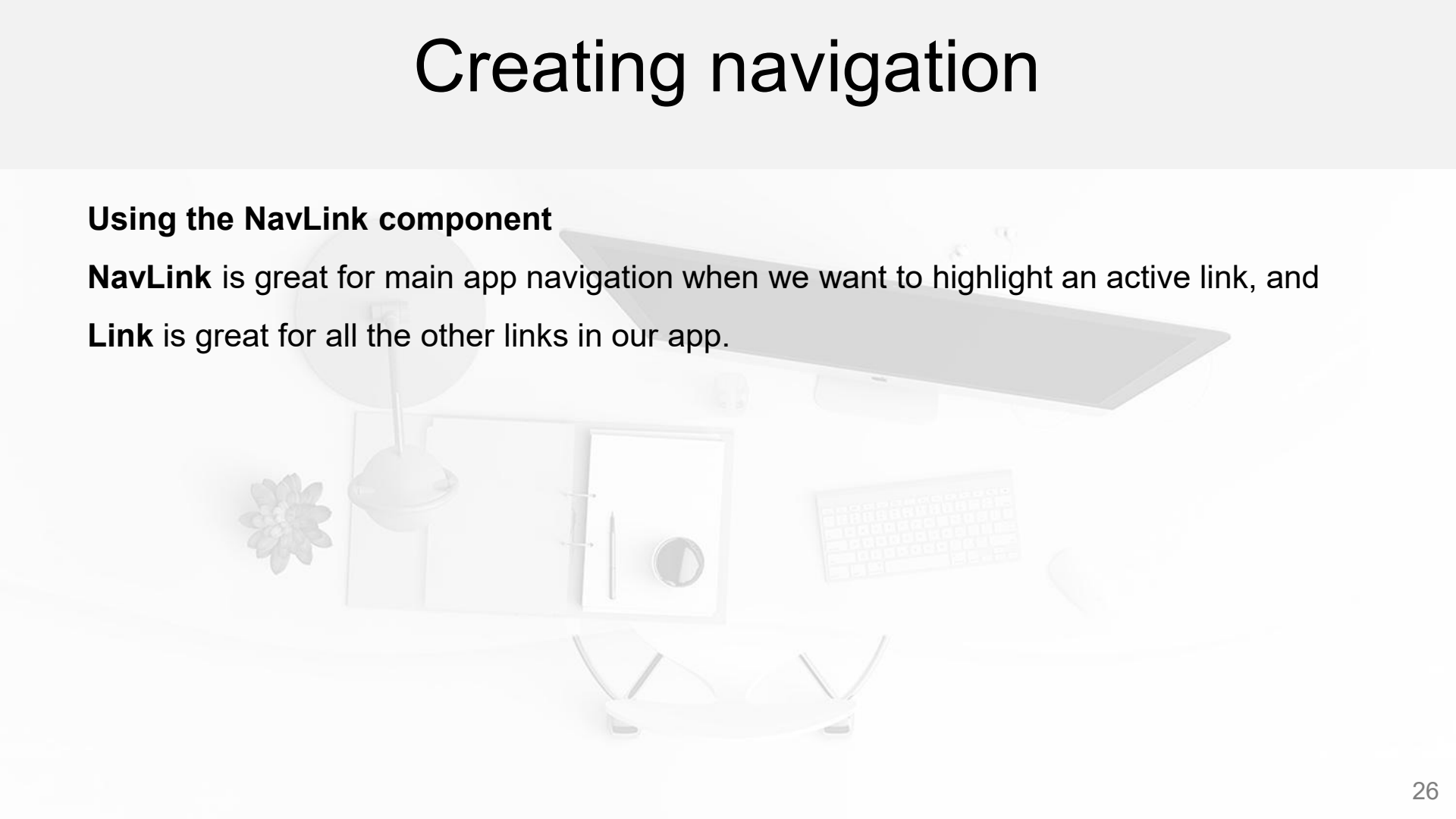
```
<NavLink
  to="products"
  className={({ isActive }) =>
    `text-white no-underline p-1 pb-0.5 border-solid border-b-2 ${
      isActive ? 'border-white' : 'border-transparent'
    }`
  }
>
  Products
</NavLink>
```

Creating navigation

Using the NavLink component

NavLink is great for main app navigation when we want to highlight an active link, and

Link is great for all the other links in our app.



Using nested routes

Understanding nested routes

- A nested route allows a segment of a route to render a component.
- Example:

Using nested routes

Understanding nested routes

- Here's what the routes for the mock-up could be:

```
const router = createBrowserRouter([
  {
    path: 'customer/:id',
    element: <Customer />,
    children: [
      {
        path: 'profile',
        element: <CustomerProfile />,
      },
      {
        path: 'history',
        element: <CustomerHistory />,
      },
      {
        path: 'tasks',
        element: <CustomerTasks />,
      },
    ],
  },
]);
```

Using nested routes

Understanding nested routes

- A **critical part** of nested routes is where **child components** are rendered in their parent.
- That is React Router's **Outlet** component.
- Example:

```
export function Customer() {
  ...
  return (
    <div>
      <Name ... />
      <Picture ... />
      <nav>
        <NavLink to="profile" ... >Profile</NavLink>
        <NavLink to="history" ... >History</NavLink>
        <NavLink to="tasks" ... >Tasks</NavLink>
      </nav>
      <Outlet />
    </div>
  );
}
```

Using nested routes

Using nested routes in the app

- We will use the **App** component for the app's shell, which renders the root path. We will then nest the products list page within this.
- Carry out the following steps:
  - Open **App.tsx** and replace all the existing content with the following:

```
import { Outlet } from 'react-router-dom';
import { Header } from './Header';

export default function App() {
  return (
    <>
      <Header />
      <Outlet />
    </>
  );
}
```

Using nested routes

Using nested routes in the app

- Carry out the following steps:
  - Open **Routes.tsx**, import the **App** component we just modified, and remove the **import** component for **Header**:

```
import {
  createBrowserRouter,
  RouterProvider,
} from 'react-router-dom';
import { ProductsPage } from './pages/ProductsPage5';
import App from './App';
```

Using nested routes

Using nested routes in the app

- Carry out the following steps:
  - Update the **router** definition

```
const router = createBrowserRouter([
  {
    path: '/',
    element: <App />,
    children: [
      {
        path: 'products',
        element: <ProductsPage />,
      }
    ]
  }
]);
```

Using nested routes

Recap

Using nested routes in the app

- Nested routes allow components to be rendered for different path segments.
- An **Outlet** component is to **render nested content** within a parent component.

Using route parameters

Understanding route parameters

- A **route parameter** is **a segment in the path** that varies.
- The value of the variable segment is available to components so that they can render something conditionally.
- Example:

  In the following path, 1234 is the ID of a customer: /customers/1234/.
- A route parameter can be defined in a route as follows:

```
{
        path: '/customer/:id',
        element: <Customer />
}
```

Using route parameters

Understanding route parameters

- Multiple route parameters can be used in a path as follows:

```
{
  path: '/customer/:customerId/tasks/:taskId',
  element: <CustomerTask />,
}
```

- Route parameter names obviously have to be **unique** within a path.

Using route parameters

Understanding route parameters

- Route parameters are available to components using React Router's **useParams** hook.

```
const params = useParams<Params>();
console.log('Customer id', params.customerId);
console.log('Task id', params.taskId);
```

- It is important to note that the **route parameter values** are **always strings** because they are extracted from paths, which are strings.

Using route parameters

Using route parameters in the app

- We will add a product page to show the description and price of each product. The path to the page will have a route parameter for the product ID.
- Carry out the following steps:
  - In the **src/pages** folder, create a file called **ProductPage.tsx** with the following import statements:

```
import { useParams } from 'react-router-dom';
import { products } from '../data/products';
```

Using route parameters

Using route parameters in the app

- Carry out the following steps:
  - Start creating the **ProductPage** component

```
type Params = {
  id: string;
};
export function ProductPage() {
  const params = useParams<Params>();
  const id =
    params.id === undefined ? undefined :
      parseInt(params.id);
}
```

Using route parameters

Using route parameters in the app

- Carry out the following steps:
  - Add a variable that is assigned to the product with the ID from the route parameter:

```
export function ProductPage() {
  const params = useParams<Params>();
  const id =
    params.id === undefined ? undefined :
      parseInt(params.id);
  const product = products.find(
    (product) => product.id === id
  );
}
```

Using route parameters

Using route parameters in the app

- Carry out the following steps:
  - Return the product information from the **product** variable in the JSX:

```
export function ProductPage() {
  ...
  return (
    <div className="text-center p-5 text-xl">
      {product === undefined ? (
        <h1 className="text-xl text-slate-900">
          Unknown product
        </h1>
      ) : (
        <>
          <h1 className="text-xl text-slate-900">
            {product.name}
          </h1>
          <p className="text-base text-slate-800">
            {product.description}
          </p>
          <p className="text-base text-slate-800">
            {new Intl.NumberFormat('en-US', {
              currency: 'USD',
              style: 'currency',
            }).format(product.price)}
          </p>
        </>
      )}
```

Using route parameters

Using route parameters in the app

- Carry out the following steps:
  - Open **Routes.tsx** and add an **import** statement for the product page:

import { ProductPage } from './pages/ProductPage';

  - Add the following highlighted route for the product page.
  - In the running app, change the browser URL to http://localhost:3000/products/2. The React Redux product should show up.

```
const router = createBrowserRouter([
  {
    path: '/',
    element: <App />,
    children: [
      {
        path: 'products',
        element: <ProductsPage />,
      },
      {
        path: 'products/:id',
        element: <ProductPage />,
      }
    ]
  }
]);
```

Using route parameters

Using route parameters in the app

- Carry out the following steps:
  - The last task is to turn the products list on the products list page into links that open the relevant product page. Open **ProductsPage.tsx** and import the **Link** component from React Router:

```
import { Link } from 'react-router-dom';
```

Using route parameters

Using route parameters in the app

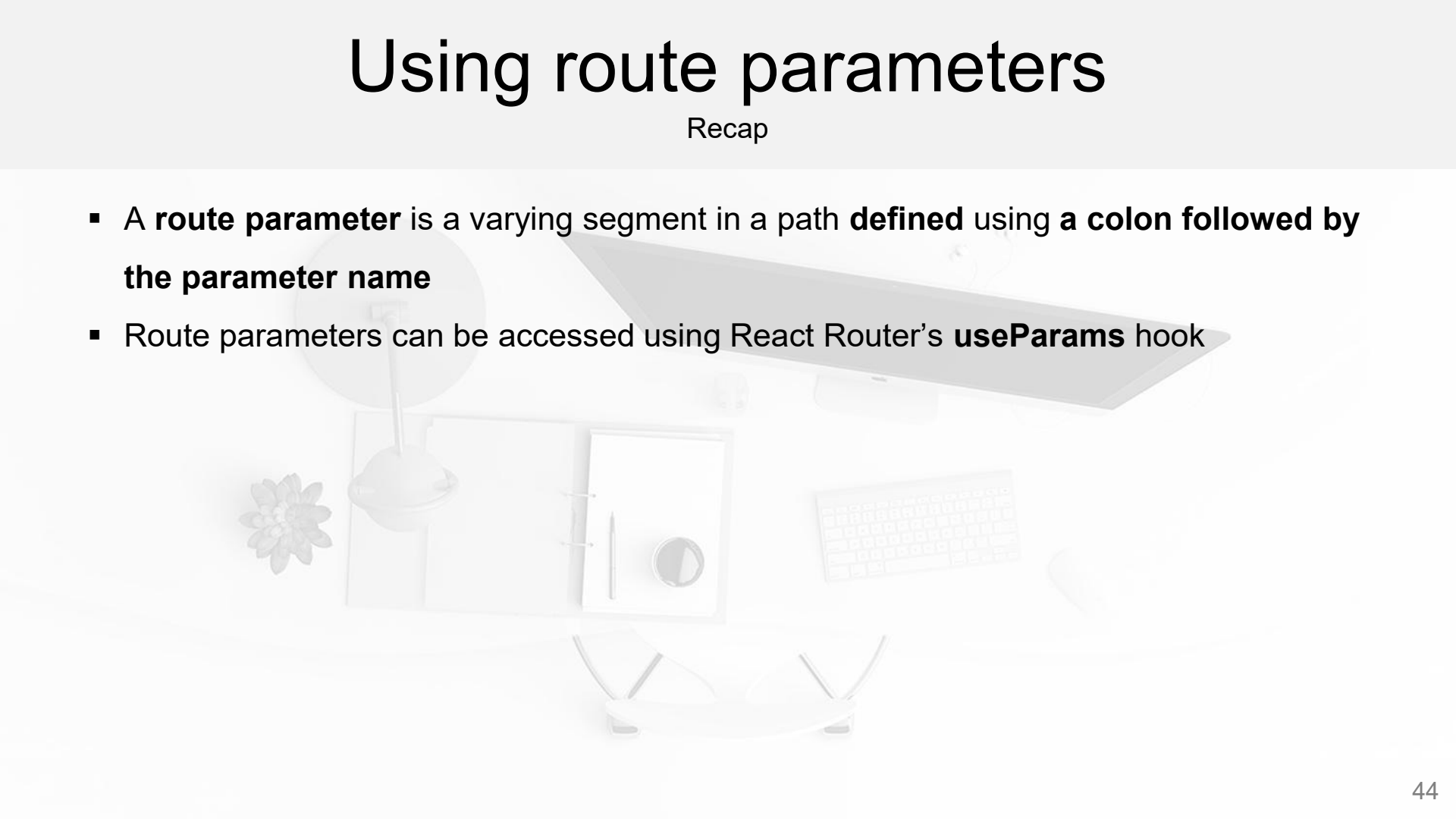
- Carry out the following steps:
  - Add a **Link** component around the product name in the JSX.
  - Return to the running app and go to the products list page. Hover over the products and you will now see that they are links.

```
<ul className="list-none m-0 p-0">
  {products.map((product) => (
    <li key={product.id}>
      <Link
        to={`${product.id}`}
        className="p-1 text-base text-slate-800
          hover:underline"
      >
        {product.name}
      </Link>
    </li>
  ))}
</ul>
```

Using route parameters

Recap

- A **route parameter** is a varying segment in a path **defined** using **a colon followed by the parameter name**
- Route parameters can be accessed using React Router's **useParams** hook



Creating an error page

Understanding error pages

- A React Router built-in error page is shown when an error occurs. Users can't easily navigate to a page that does exist.
- The following is an example of a custom error page defined for a customer's route:

```
const router = createBrowserRouter([
  ...,
  {
    path: 'customers',
    element: <CustomersPage />,
    errorElement: <CustomersErrorPage />
  },
  ...
]);
```

Creating an error page

Adding an error page

- Carry out the steps below to create an error page in the app:
  - Create a new page called **ErrorPage.tsx** in the **src/pages** folder

```
import { Header } from '../Header';

export function ErrorPage() {
  return (
    <>
      <Header />
      <div className="text-center p-5 text-xl">
        <h1 className="text-xl text-slate-900">
          Sorry, an error has occurred
        </h1>
      </div>
    </>
  );
}
```

Creating an error page

Adding an error page

- Carry out the steps below to create an error page in the app:
  - Open **Routes.tsx** and add an **import** statement for the error page:

    import { ErrorPage } from './pages/ErrorPage';

  - Specify the error page on the root route:
  - Switch back to the running app and change the browser URL to http://localhost:3000/invalid.

    The error page will be shown.

```
const router = createBrowserRouter([
  {
    path: '/',
    element: <App />,
    errorElement: <ErrorPage />,
    children: ...
  },
]);
```

Creating an error page

Adding an error page

- Carry out the steps below to create an error page in the app:
  - Improve it by providing the user with more information, which we can get from React Router's **useRouteError** hook. Open **ErrorPage.tsx** again and add an **import** statement for **useRouteError**:

    import { useRouteError } from 'react-router-dom';

  - Assign the error to an **error** variable before the component's return statement using **useRouteError**

```
export function ErrorPage() {
  const error = useRouteError();
  return ...
}
```

Creating an error page

Adding an error page

- Carry out the steps below to create an error page in the app:
  - The **error** variable is of the **unknown** type. Add the following type predicate function beneath the component:

```
function isError(error: any): error is { statusText:
string } {
  return "statusText" in error;
}
```

Creating an error page

Adding an error page

- Carry out the steps below to create an error page in the app:
  - Use this function to render the information in the **statusText** property
  - In the running app, the information about the error appears on the error page as an invalid path

```
return (
  <>
    <Header />
    <div className="text-center p-5 text-xl">
      <h1 className="text-xl text-slate-900">
        Sorry, an error has occurred
      </h1>
      {isError(error) && (
        <p className="text-base text-slate-700">
          {error.statusText}
        </p>
      )}
    </div>
  </>
);
```

Creating an error page

Recap

Adding an error page

- Use an **errorElement prop** on a route **to catch and display errors**.
- **Specific error information** can be obtained using the **useRouteError** hook.

Using index routes

Understanding index routes

- An **index route** can be thought of as **a default child route**.
- In React Router, if no children match a parent route, it will display an index route if one is defined.
- An index route has **no path** and instead has an **index Boolean property**, as in the following example:

```
{
  path: "/",
  element: <App />,
  children: [
    {
      index: true,
      element: <HomePage />,
    },
    ...,
  ]
}
```

Using index routes

Using an index route in the app

- Carry out the following steps:
  - Create a new file in the **src/pages** folder called **HomePage.tsx** with the following

```
export function HomePage() {
  return (
    <div className="text-center p-5 text-xl">
      <h1 className="text-xl text-slate-900">Welcome to
        React Tools!</h1>
    </div>
  );
}
```

Using index routes

Using an index route in the app

- Carry out the following steps:
  - Open **Routes.tsx** and import the home page:

```
import { HomePage } from './pages/HomePage';
```

  - Add the home page as an index page for the root path:

```
const router = createBrowserRouter([
  {
    path: '/',
    element: <App />,
    errorElement: <ErrorPage />,
    children: [
      {
        index: true,
        element: <HomePage />,
      },
      ...
    ]
  }
]);
```

Using index routes

Using an index route in the app

- Carry out the following steps:
  - Add links to the logo and app title in the header to go to the home page. Open **Header.tsx** and import the **Link** component from React Router

```
import { NavLink, Link } from 'react-router-dom';
```

Using index routes

Using an index route in the app

- Carry out the following steps:
  - Wrap links to the root page around the logo and title.
  - In the running app, click the app title to go to the root page, and you will see the welcome message displayed

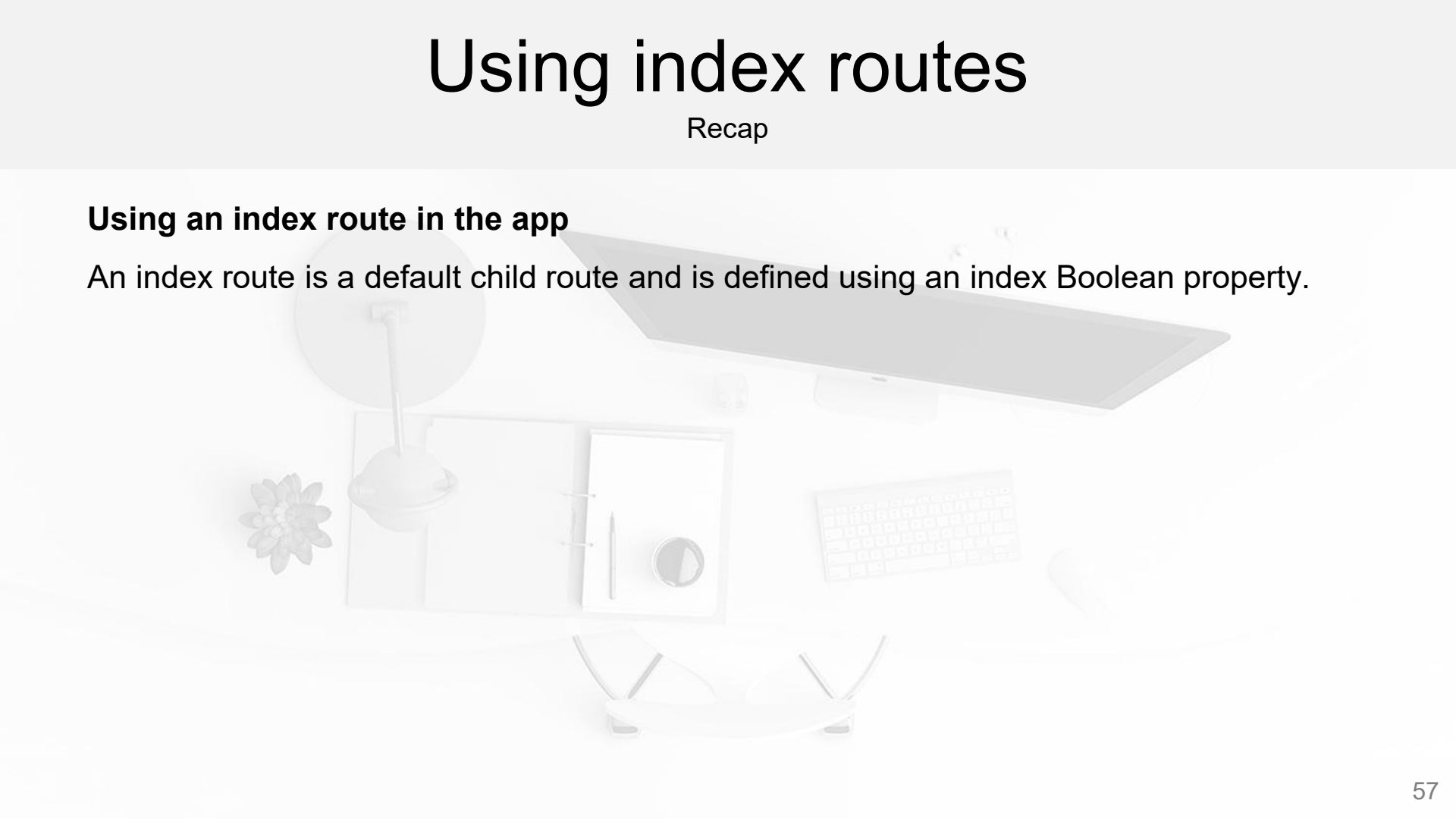
```
<header ...>
  <Link to="">
    <img src={logo} ... />
  </Link>
  <Link to="">
    <h1 ...>React Tools</h1>
  </Link>
  <nav>
    ...
  </nav>
</header>
```

Using index routes

Recap

Using an index route in the app

An index route is a default child route and is defined using an index Boolean property.



Using search parameters

Understanding search parameters

- **Search parameters** are part of a URL that comes **after the ? character** and **separated by the & character**.
- Search parameters are sometimes referred to as **query parameters**.
- In the following URL, type and when are search parameters: https://somewhere.com/?type=sometype&when=recent.

Using search parameters

Understanding search parameters

- React Router has a hook that returns functions for getting and setting search parameters called **useSearchParams**:

```
const [searchParams, setSearchParams] = useSearchParams();
```

- **searchParams** is a JavaScript URLSearchParams object. There is a **get** method on URLSearchParams, which can be used to get the value of a search parameter.
- The following example gets the value of a search parameter called type:

```
const type = searchParams.get('type');
```

Using search parameters

Understanding search parameters

- **setSearchParams** is a function used to set search parameter values. The function parameter is an object as in the following example:

```
setSearchParams({ type: 'sometype', when: 'recent' });
```

Using search parameters

Adding search functionality to the app

- **Description:** We will **add a search box** to the header of the app. Submitting a search will take the user to the products list page and list a filtered set of products matching the search criteria.
- Carry out the following steps:
  - Open **Header.tsx** and add **useSearchParams** to the React Router import. Also, add an import statement for the **FormEvent** type from React:

```
import { FormEvent } from 'react';
import {
  NavLink,
  Link,
  useSearchParams
} from 'react-router-dom';
```

Using search parameters

Adding search functionality to the app

- Carry out the following steps:
  - Use the **useSearchParams** hook to destructure functions to get and set search parameters before the **return** statement:

```
export function Header() {
  const [searchParams, setSearchParams] =
useSearchParams();
  return ...
}
```

Using search parameters

Adding search functionality to the app

- Carry out the following steps:
  - Add the following search form above the logo:

```
<header ...>
    <form
        className="relative text-right"
        onSubmit={handleSearchSubmit}
    >
        <input
            type="search"
            name="search"
            placeholder="Search"
            defaultValue={searchParams.get('search') ?? ''}
            className="absolute right-0 top-0 rounded py-2 px-3
            text-gray-700"
        />
    </form>
    <Link to="">
        <img src={logo} ... />
    </Link>
    ...
</header>
```

Using search parameters

Adding search functionality to the app

- Carry out the following steps:
  - Add a **handleSearchSubmit** function as follows, just above the return statement:

```
export function Header() {
  const [searchParams, setSearchParams] =
    useSearchParams();
  function handleSearchSubmit(e:
    FormEvent<HTMLFormElement>) {
    e.preventDefault();
    const formData = new FormData(e.currentTarget);
    const search = formData.get('search') as string;
    setSearchParams({ search });
  }
  return ...
}
```

Using search parameters

Adding search functionality to the app

- Carry out the following steps:
  - Need to filter the products list with the value of the search parameter. Open **ProductsPage.tsx** and add **useSearchParams** to the import statement:

    import { Link, useSearchParams } from 'react-router-dom';
  - At the top of the **ProductsPage** component, destructure **searchParams** from **useSearchParams**:

```
export function ProductsPage() {
  const [searchParams] = useSearchParams();
  return ...
}
```

Using search parameters

Adding search functionality to the app

- Carry out the following steps:
  - Add the following function just before the return statement to filter the products list by the search value:

```
const [searchParams] = useSearchParams();
function getFilteredProducts() {
    const search = searchParams.get('search');
    if (search === null || search === "") {
        return products;
    } else {
        return products.filter(
            (product) =>
            product.name
            .toLowerCase()
            .indexOf(search.toLowerCase()) > -1
        );
    }
}
return ...
```

Using search parameters

Adding search functionality to the app

- Carry out the following steps:
  - Use the function we just created in the JSX to output the filtered products. Replace the reference to products with a call to **getFilteredProducts**.
  - In the running app, whilst on the home page, enter some search criteria in the search box and press Enter to submit the search.

```
<ul className="list-none m-0 p-0">
  {getFilteredProducts().map((product) => (
    <li
      key={product.id}
      className="p-1 text-base text-slate-800"
    >
      <Link
        to={`${product.id}`}
        className="p-1 text-base text-slate-800
          hover:underline"
      >
        {product.name}
      </Link>
    </li>
  ))}
</ul>
```

Using search parameters

Recap

Adding search functionality to the app

- The **useSearchParams** hook from React Router allows you to set and get URL search parameters.
- The parameters are also structured in a JavaScript URLSearchParams object.

Navigating programmatically

Carry out the following steps:

- Open **Header.tsx** and add import the **useNavigate** hook from React Router:

```
import {
  NavLink,
  Link,
  useSearchParams,
  useNavigate
} from 'react-router-dom';
```

Navigating programmatically

Carry out the following steps:

- Invoke **useNavigate** after the call to the **useSearchParams** hook. Assign the result to a variable called navigate:

```
export function Header() {
  const [searchParams, setSearchParams] =
    useSearchParams();
  const navigate = useNavigate();
  ...
}
```

The navigate variable is a function that can be used to navigate. It takes in an argument for the path to navigate to.

Navigating programmatically

Carry out the following steps:

- In **handleSearchSubmit**, replace the **setSearchParams** call with a call to navigate to go to the products list page with the relevant search parameter:

```
function handleSearchSubmit(e: FormEvent<HTMLFormElement>)
{
  e.preventDefault();
  const formData = new FormData(e.currentTarget);
  const search = formData.get('search') as string;
  navigate(`/products/?search=${search}`);
}
```

Navigating programmatically

Carry out the following steps:

- Remove **setSearchParams** from the **useSearchParams** call:

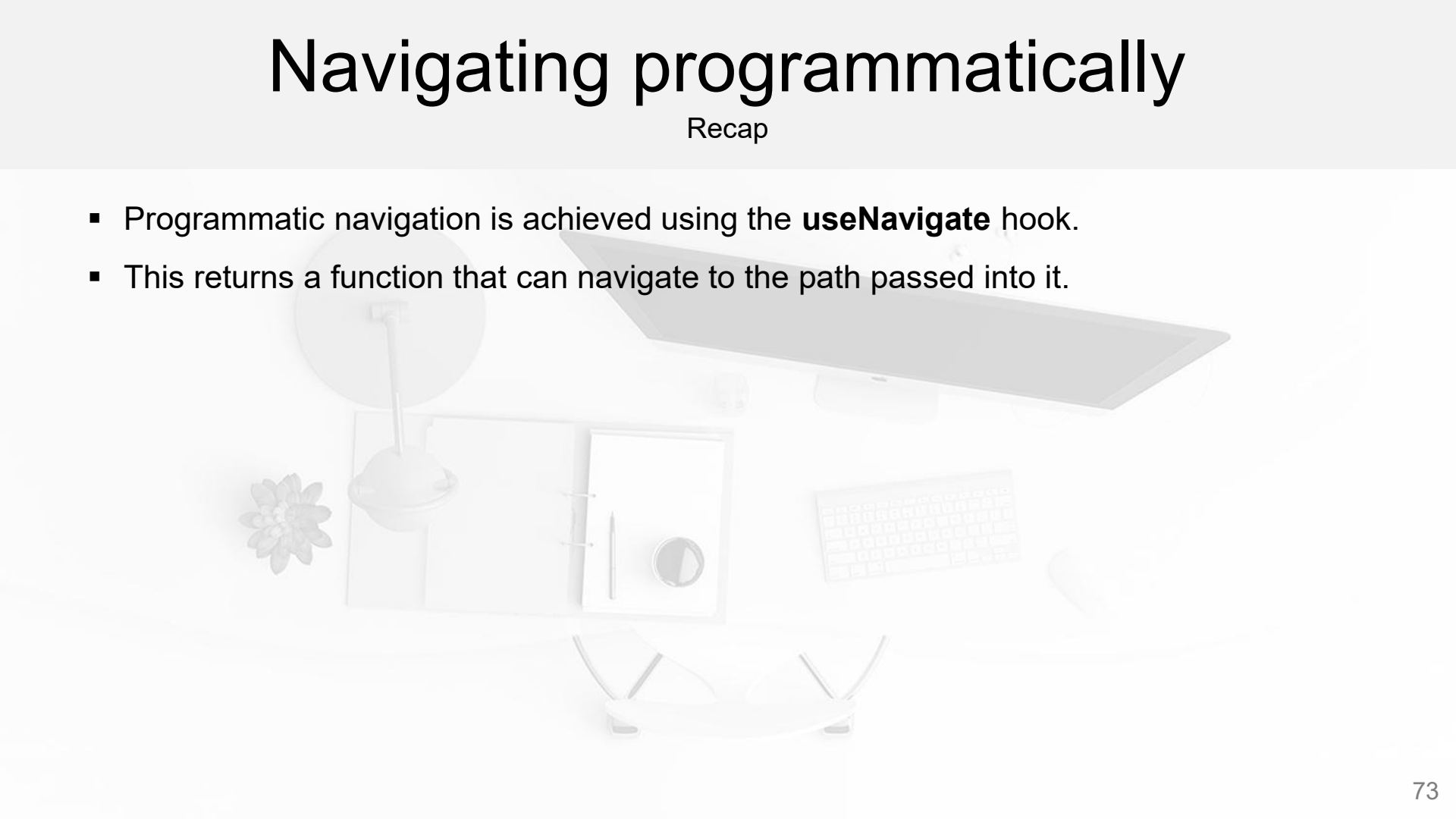
```
const [searchParams] = useSearchParams();
```

- In the running app, enter some search criteria in the search box and press **Enter** to submit the search.

Navigating programmatically

Recap

- Programmatic navigation is achieved using the **useNavigate** hook.
- This returns a function that can navigate to the path passed into it.



Using form navigation

- Use **React Router's Form** component to navigate to the products list page when the search criteria are submitted.
- Form is a wrapper around the HTML form element that handles the form submission on the client side. This will replace the use of **useNavigate** and simplify the code.
- Carry out the following steps:
  - In **Header.tsx**, start by removing **useNavigate** from import for the React Router and replace it with the **Form** component:

```
import {
  NavLink,
  Link,
  useSearchParams,
  Form
} from 'react-router-dom';
```

Using form navigation

- Carry out the following steps:
  - In the JSX, replace the form element with **React Router's Form** component:

```
<Form
    className="relative text-right"
    onSubmit={handleSearchSubmit}
>
    <input ... />
</Form>
```

Using form navigation

- Carry out the following steps:
  - In the **Form** element in the JSX, remove the **onSubmit** handler. Replace this with the following **action attribute** so that the form is sent to the products route:

```
<Form
  className="relative text-right"
  action="/products"
>
  ...
</Form>
```

Using form navigation

- Carry out the following steps:
  - The remaining tasks are to remove the following code:
    - Remove the React import statement because **FormEvent** is redundant now
    - Remove the call to **useNavigate** because this is no longer required
    - Remove the **handleSearchSubmit** function because this is no longer required
  - In the running app, enter some search criteria in the search box and press Enter to submit the search.

Implementing lazy loading

Understanding React lazy loading

- By default, all React components are bundled together and loaded when the app first loads. → This is inefficient for large apps.
- **Lazily loading** React components addresses this issue because lazy components aren’t included in the initial bundle that is loaded; instead, their JavaScript is fetched and loaded when rendered.

Implementing lazy loading

Understanding React lazy loading

- There are **two main steps** to lazy loading React components:
  - First, **the component must be dynamically imported**:

```
const LazyPage = lazy(() => import('./LazyPage'));
```

lazy is a function from React that enables the imported component to be lazily loaded. **Note** that the **lazy page must be a default export**.

Implementing lazy loading

Understanding React lazy loading

- There are **two main steps** to lazy loading React components:
  - The second step is to **render the lazy component inside React's Suspense component**:

```
<Route
  path="lazy"
  element={
    <Suspense fallback={<div>Loading…</div>}>
      <LazyPage />
    </Suspense>
  }
/>
```

The Suspense component's **fallback prop** can be set to an element to render while the lazy page s being fetched.

Implementing lazy loading

Adding a lazy admin page to the app

- Carry out the following steps:
  - Create a file called **AdminPage.tsx** in the **src/pages** folder with the following content:

```
export default function AdminPage() {
  return (
    <div className="text-center p-5 text-xl">
      <h1 className="text-xl text-slate-900">Admin Panel</h1>
      <p className="text-base text-slate-800">You shouldn't come here
      often because I'm lazy</p>
    </div>
  );
}
```

Implementing lazy loading

Adding a lazy admin page to the app

- Carry out the following steps:
  - Open **Routes.tsx** and import **lazy** and **Suspense** from React:

```
import { lazy, Suspense } from ‘react’;
```

  - Import the admin page as follows (it is important that this **comes after all the other import statements**):

```
const AdminPage = lazy(() => import('./pages/AdminPage’));
```

Implementing lazy loading

Adding a lazy admin page to the app

- Carry out the following steps:
  - Add the **admin** route as follows:

```
const router = createBrowserRouter([
  {
    path: '/',
    element: <App />,
    errorElement: <ErrorPage />,
    children: [
      ...,
      {
        path: 'admin',
        element: (
        <Suspense
          fallback={
            <div className="text-center p-5 text-xl
            text-slate-900">
            Loading...
            </div>
          }
        >
        <AdminPage />
        </Suspense>
        )
      }
    ]
  }
]);
```

Implementing lazy loading

Adding a lazy admin page to the app

- Carry out the following steps:
  - Open **Header.tsx** and add a link to the admin page after the **Products** link

```
<nav>
  <NavLink ... >
    Products
  </NavLink>
  <NavLink
    to="admin"
    className={({ isActive }) =>
      `text-white no-underline p-1 pb-0.5 border-solid border-b-2 ${
        isActive ? 'border-white' : 'border-transparent'
      }`
    }
  >
    Admin
  </NavLink>
</nav>
```

Implementing lazy loading

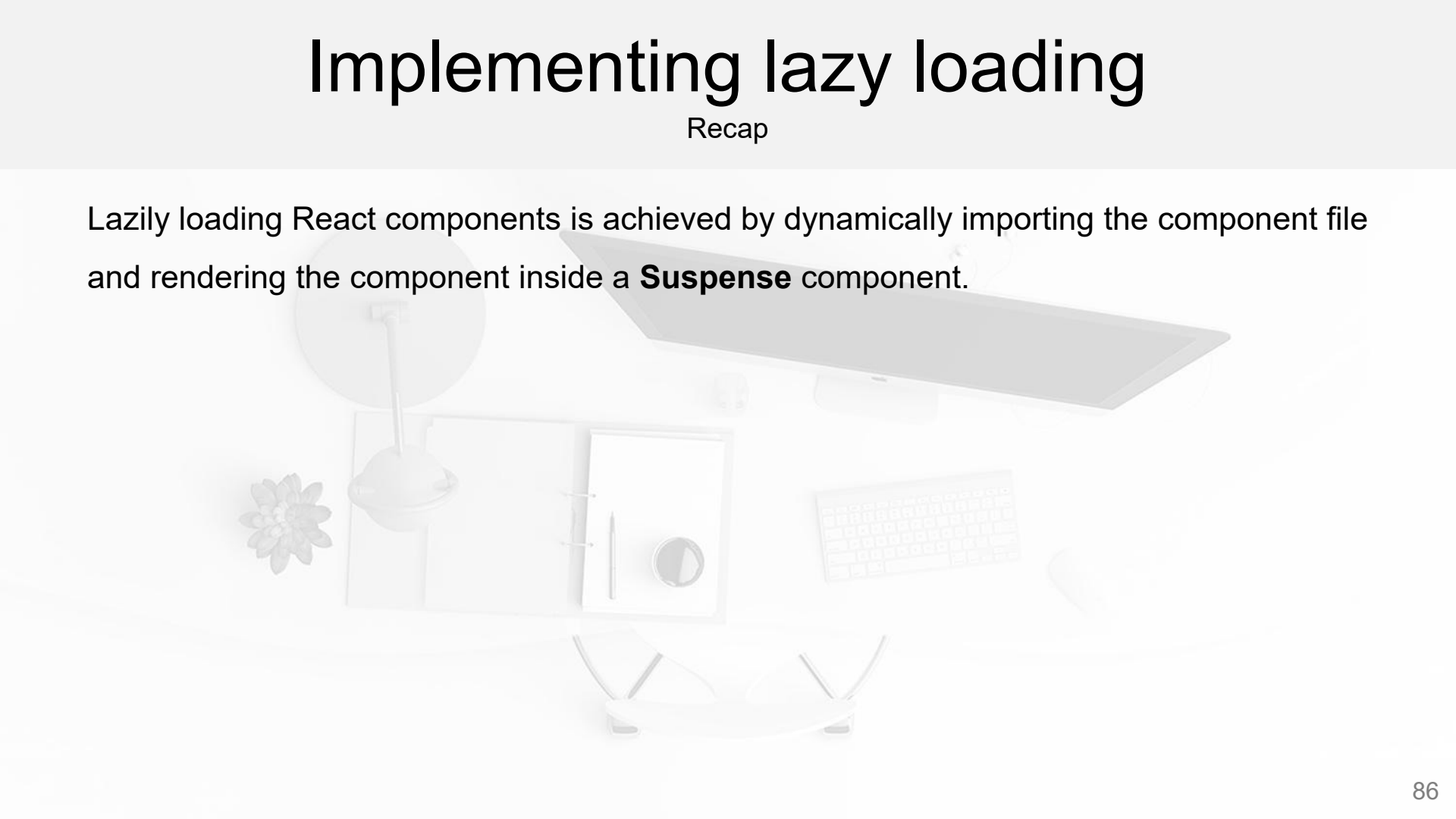
Adding a lazy admin page to the app

- Carry out the following steps:
  - In the running app, open the browser **DevTools** and go to the **Network** tab and clear out any existing requests. Slow down the connection by selecting **Slow 3G** from the **No throttling** menu.
  - Click on the **Admin** link in the header. The loading indicator appears because the JavaScript for the admin page is being downloaded.

Implementing lazy loading

Recap

Lazily loading React components is achieved by dynamically importing the component file and rendering the component inside a **Suspense** component.



Summary

- **createBrowserRouter** is used to define all web app's routes. A route contains a path and a component to render when the path matches the browser URL.
- **errorElement** prop is used for a route to render a custom error page.
- **Nested routes** allows the App component to render the app shell and page components within it. React Router's **Outlet** component is used inside the App component to render page content. **Index route** on the root route is used to render a welcome message.
- React Router's **NavLink** component is used to render navigation links that are highlighted when their route is active. The **Link** component is great for other links that have static styling requirements.

Summary

- React Router's **Form** component is used to navigate to the products list page when the search form is submitted.
- Route parameters and search parameters allow parameters to be passed into components so that they can render dynamic content. **useParams** gives access to route parameters, and **useSearchParams** provides access to search parameters.
- React components can be lazily loaded to increase startup performance. This is achieved by dynamically importing the component file and rendering the component inside a **Suspense** component.


Q&A